MEMORIA DE LA PRÁCTICA:

Aplicación profesional (DICOM)

Escenarios

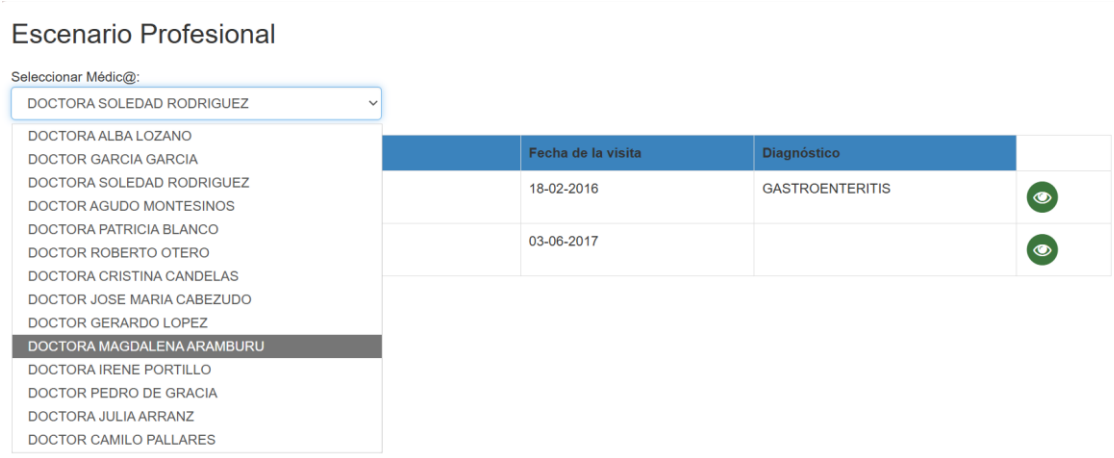


Figura 1. Selector de médicos.



Figura 2. Lista de pacientes asignados a un médico.

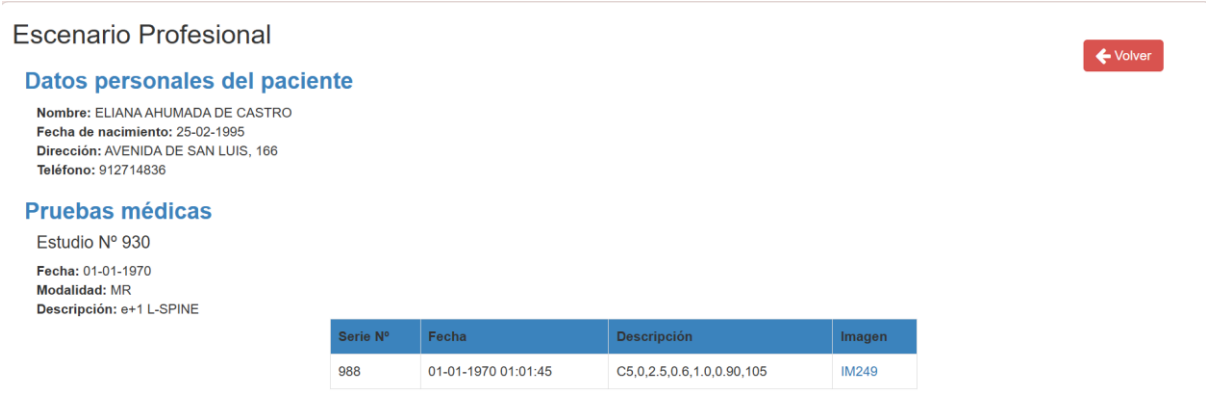


Figura 3. Datos e imágenes asociados a un paciente.

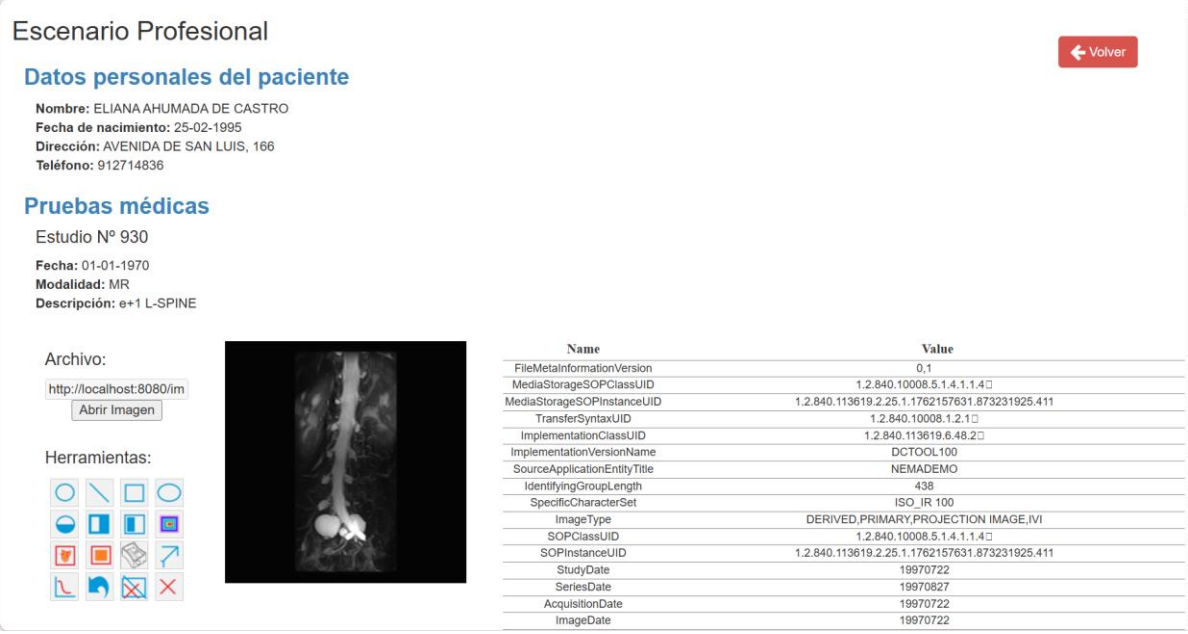


Figura 4. Imagen de un paciente.

Problemas encontrados

Durante el desarrollo de la parte obligatoria de esta práctica, se presentaron varios desafíos técnicos centrados principalmente en la compatibilidad de versiones y la visualización de la interfaz. El primer obstáculo fue un error de tipo TypeError relacionado con la propiedad last-modified, causado por una incompatibilidad entre la versión de Node.js instalada (v24) y las librerías antiguas requeridas por el proyecto. Este problema se solventó mediante la recuperación y el uso de las carpetas de dependencias (node_modules) de la práctica anterior, asegurando así un entorno de ejecución estable.

Posteriormente, se detectaron fallos en el arranque de la aplicación AngularJS debido a errores de inyección de dependencias ($injector:moduler). Esto se debió a discrepancias entre el orden de carga de los scripts en el archivo index.html y la declaración de módulos aún no implementados en el controlador principal. Una vez estabilizada la aplicación, la interfaz presentaba problemas estéticos significativos: los iconos de las herramientas DICOM se desalineaban y no mostraban sus dibujos completos. Estos errores visuales se corrigieron mediante ajustes en el archivo style.css, optimizando el ancho de las columnas, aplicando propiedades de visualización bloque (inline-block) y ajustando el tamaño del fondo (background-size) de los botones.

Finalmente, la tabla de metadatos DICOM no se visualizaba correctamente a pesar de que el servidor enviaba los datos. El problema residía en las condiciones impuestas por la directiva ng-show="imageData==1". La solución definitiva consistió en refactorizar esta lógica para condicionar la visibilidad de las cabeceras a la existencia real de datos en el objeto (Tag.length > 0), permitiendo así una renderización fluida de la información técnica asociada a las imágenes médicas.

Modificaciones

Mejoras del aspecto

- **Cabecera más estética**:
  En index.html se ha escrito un <header> más elaborado, que incluye un icono y el nombre del centro de médico donde se emplea la aplicación (al que se ha denominado “Hospital Digital UPM”), así como un título (“Gestión de imagen médica”) y un subtítulo (“Acceso restringido a profesionales”). Además, se han añadido reglas al archivo style.css para ajustar los colores de la cabecera, la tipografía y el tamaño de las letras.
  Para que la aplicación tenga un estilo unificado, se han cambiado los colores de las cabeceras de la tabla de pacientes por tonos morados.



Figura 5. Cabecera mejorada.

- **Interacciones y estilo de los botones**:

  Se han implementado diversas mejoras de interactividad y diseño visual mediante la edición de la hoja de estilos style.css y la estructura index.html. En el archivo CSS, se ha incorporado la pseudo-clase ‘:hover’ para aplicar un cambio de color de fondo en las filas de la tabla de visitas, utilizando el selector ‘:not(#headerTable)’ para garantizar que la cabecera morada permanezca estática y mantener la consistencia visual.

  Los botones de acción principal, como "visualizar" y "volver", incluyen ahora animaciones basadas en la propiedad ‘transform: scale()’, lo que permite aumentar su tamaño durante la interacción. Se ha seleccionado una tonalidad verde-lima para estos elementos, para facilitar la identificación de funciones mediante un alto contraste cromático. Mientras que el estilo del botón "visualizar" se gestionó mediante atributos directos en el HTML sobre el icono de Font Awesome, el botón "volver" se personalizó en el CSS a través de su identificador #comeBackButton, corrigiendo su posicionamiento mediante un ajuste de ‘margin-top’.

  Se ha estandarizado el aspecto del botón "Abrir Imagen" empleando las clases ‘btn btn-primary' de Bootstrap, añadiendo efectos de sombra ‘box-shadow’ y transiciones suaves para aportar una sensación de fluidez a la herramienta.

Finalmente, con el fin de proporcionar un feedback visual inmediato al profesional médico, se han implementado efectos dinámicos en la barra de herramientas del visor DICOM. Al interactuar con los botones, se activa un marco de color rojo que destaca la herramienta seleccionada, facilitando la identificación visual de la función activa. En el caso específico del botón de cierre, se ha aplicado una transformación de escala que hace que el botón se minimice levemente al ser pulsado, confirmando la finalización de la sesión de visualización. En el CSS se ha utilizado la propiedad 'transform: scale()' para lograr la reducción física del botón, mientras que el aura roja se ha generado mediante 'box-shadow' dinámicos. En el controlador, se inyectó el servicio $timeout para gestionar la variable $scope.isClosing, permitiendo que la directiva 'ng-class' mantenga los efectos visuales y el mensaje de cierre durante 500ms antes de destruir el visor.



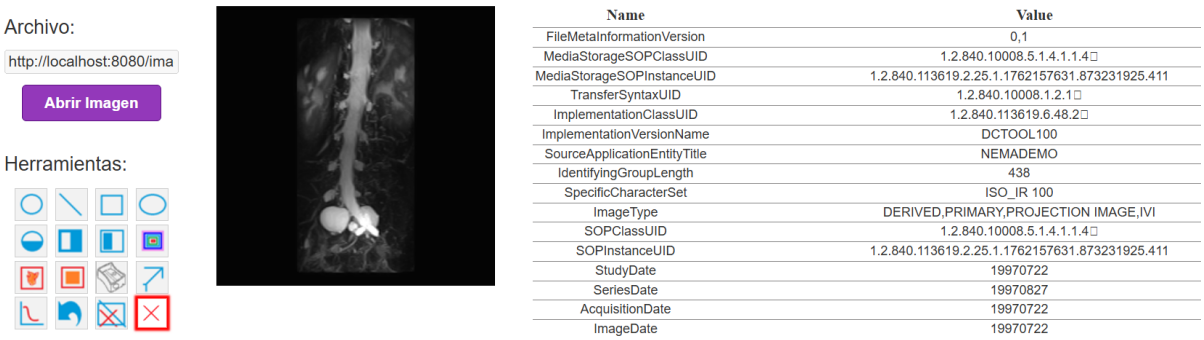
Figura 6. Agrandamiento y efecto brillo en el botón visualizar. Sombreado sobre la fila seleccionada.



Figura 7. Agrandamiento y efecto brillo en el botón volver.



Figura 8. Agrandamiento y efecto sombra en el botón Abrir Imagen.



Name	Value
FileMetaInformationVersion	0,1
MediaStorageSOPClassUID	1.2.840.10008.5.1.4.1.1.4
MediaStorageSOPInstanceUID	1.2.840.113619.2.25.1.1762157631.873231925.411
TransferSyntaxUID	1.2.840.10008.1.2.1
ImplementationClassUID	1.2.840.113619.6.48.2
ImplementationVersionName	DCTOOL100
SourceApplicationEntityTitle	NEMADEMO
IdentifyingGroupLength	438
SpecificCharacterSet	ISO_IR 100
ImageType	DERIVED,PRIMARY,PROJECTION IMAGE,IVI
SOPClassUID	1.2.840.10008.5.1.4.1.1.4
SOPInstanceUID	1.2.840.113619.2.25.1.1762157631.873231925.411
StudyDate	19970722
SeriesDate	19970827
AcquisitionDate	19970722
ImageDate	19970722

Figura 9. Marco en los botones de herramientas.

Mejoras de la usabilidad general

- **Feedback de carga**:
  Se ha implementado un mensaje de carga que evita que el profesional sanitario piense que la aplicación se ha colgado mientras espera la respuesta del servidor o la BBDD DICOM. Esta mejora favorece la prevención de errores, ya que evita que el usuario haga múltiples clics mientras se procesa la información.
  La implementación se ha llevado a cabo modificando los ficheros app.js, index.html y style.css. En primer lugar, se han modificado las funciones doctorVisits() y selectedPatient() del controlador, inicializando una variable '$scope.loading', que se activa y desactiva al inicio y al final de cada función de consulta. En el HTML se ha añadido el mensaje de carga, empleando la clase 'fa-spin' de Font Awesome que proporciona una indicación visual de movimiento que confirma que el sistema está trabajando. Por último, en el CSS se ha creado una capa que bloquea la interacción mientras carga y se han establecido los estilos del mensaje.



Figura 9. Mensaje de carga.

Nuevas funcionalidades

- **Buscador de pacientes**:

La implementación de un buscador permite al profesional sanitario filtrar y localizar rápidamente a un paciente específico dentro de un listado extenso sin necesidad de realizar desplazamientos manuales. Esta utilidad optimiza el flujo de trabajo en la consulta, reduciendo la carga cognitiva y el tiempo de respuesta al gestionar las visitas asignadas.

En el HTML se ha integrado un campo de entrada (input) vinculado a la directiva ng-model="searchPatient", que captura el texto introducido por el usuario en tiempo real. Para ejecutar el filtrado, se aplicó el filtro nativo de AngularJS dentro de la directiva 'ng-repeat' de la tabla de visitas (visit in visits | filter:searchPatient). Para el estilo se ha definido un contenedor (search-container) que centra el buscador y lo separa visualmente de la tabla mediante márgenes adecuados. Además, se ha añadido un borde violeta y efectos de iluminación (box-shadow), que se activan cuando el usuario ubica el ratón en el buscador para comenzar a escribir.



Figura 10. Buscador de pacientes.